

Abbreviated Lambdas for Fun and Profit

Contents

1. [Motivation](#)
2. [Proposal](#)
3. [Examples](#)
4. [Hyper-abbreviated lambdas](#)
5. [Prior Work and Effects on Existing Code](#)
6. [Revision History](#)
7. [Acknowledgements and References](#)

1. Motivation

The introduction of lambdas in C++11 transformed the landscape of C++ tremendously by allowing users to write arbitrary functions, predicates, and callbacks in-situ. Suddenly, instead of having to write function objects somewhere far from the code, we could write them inline. C++14 improved lambdas both by expanding their functionality (with generic lambdas) and by making them shorter (by being able to use `auto` instead of spelling out the full type name of the parameters).

While lambdas are at the point where nearly arbitrary functionality is possible, it is still cumbersome and at times unnecessarily verbose to write out lambdas in the simplest of cases. A disproportionate number of lambda expressions are extremely short, consisting solely of a single expression in the body. These simple lambdas typically occur in one (or more) of these situations:

- Unary/Binary predicates into algorithms.
- Callbacks
- Binding - simply reordering or adding new arguments for other functions. Basically, a better `std::bind`.
- Lifting - taking overloaded functions and lifting them into a generic lambda so that they can be passed into a function.

The main idea linking all of these situations is that we're trying to lift an *expression* into a function. That expression isn't merely an implementation detail: it's the most important detail. The goal as a programmer is to, as transparently as possible, make the expression we want to use available as a function in the context we want to use it. But in all these cases, the amount of code necessary to write this transparent lambda is excessive compared to the actual functionality being expressed.

A perfect example of this excessive and complicated code necessary to express a seemingly simple idea is the lifting of overload sets [1]. We cannot simply write:

```
template <class T>
T twice(T x) { return x + x; }

template <class I>
void f(I first, I last) {
    transform(first, last, twice); // error
}
```

We need to wrap `twice` in a lambda. But the correct way to do that is quite complex. We need to keep lots of details in mind. First, we of course need a generic lambda:

```
[](auto x) { return twice(x); }
```

But then, this incurs unnecessary copies in and out, so we would want to forward into and out of the lambda:

```
[](auto&& x) { return twice(std::forward<decltype(x)>(x)); }
```

But now, if `twice` returned a reference type, the lambda drops that. In order to mirror that properly, we would need a trailing return type:

```
[](auto&& x) -> decltype(auto) { return twice(std::forward<decltype(x)>(x)); }
```

The problems with this approach are more subtle, but no less real.

1.1. SFINAE and `noexcept`

The motivation is to *transparently* wrap an expression to make it usable as a function. If we have an overload set, like:

```
int& foo(int&) noexcept;
std::string foo(char const* );
```

We can use `foo` in generic code directly by name and query whether it's invocable with a given argument type and query whether it's `noexcept` with a given argument type. This is *prima facie* important and useful. If we naively wrap `foo` in a lambda that simply returns:

```
[](auto&& arg) -> decltype(auto) { return foo(arg); }
```

Then we immediately lose both abilities. This lambda is *never* `noexcept`. Since the user took the effort and deliberately marked `foo(int&)` as being `noexcept`, we lose that information if we just drop it on the floor. This lambda also advertises itself as being invocable with any argument type, which can lead to problems.

With only a single expression, there is no advantage to writing `decltype(auto)` over `decltype(foo(arg))`. But in certain contexts, preferring the latter can be the difference between a program that compiles and does the expected thing - and a program that fails to compile.

```
bool call_me_maybe(function<bool(int)> ); // #1
bool call_me_maybe(function<bool(string)> ); // #2

// non-SFINAE-friendly lambda
call_me_maybe([](auto x) { return x == 2; }); // error! lots and lots of diagnostic

// SFINAE-friendly lambda
call_me_maybe([](auto x) -> decltype(x == 2) { return x == 2; }); // OK, calls #1
```

The intent of the programmer is simple, and the reason for the compile failure of the first attempt is quite complex (attempting to instantiate `function<bool(string)>` involves instantiating the particular `operator()` of the lambda to determine its return type, but this instantiation isn't considered in the immediate context and so this failure is a hard error rather than a substitution failure). The technique to fix this, the trailing

`decltype`, is an advanced technique - far more advanced than the problem seems to call for.

1.2. Does Concepts solve the problem?

With Concepts, we have a better, clearer way to constrain templates. It's tempting to believe that the introduction of Concepts to C++2a would obviate us of this problem. But Concepts only helps us if we help Concepts.

1.2.1. Concept error messages

Let's consider a constrained implementation of `std::find_if`, where we only focus on the predicate constraint:

```
template <class F, <class... Args>
concept Predicate = requires(F f, Args... args) {
    { std::invoke(f, args...) } -> bool;
};

template <class Iter, Predicate<typename std::iterator_traits<Iter>::reference F>
Iter find_if(Iter, Iter, F f);
```

Now, what happens when we try to invoke this incorrectly?

```
std::vector<std::string> vs;
find_if(vs.begin(), vs.end(), [](auto const& s) { return s == 2; }); // 1: Concept unfriendly
find_if(vs.begin(), vs.end(), [](auto const& s) -> decltype(s == 2) { return s == 2; }); // 2: Concept friendly
```

Both invocations are erroneous. However, the non-Concept-friendly lambda generates [240 lines of errors](#), going through all the various, irrelevant overloads of `operator==` that don't work.

On the other hand, the second invocation, the Concept-friendly lambda, produces a short message clearly indicating that the lambda does not model the `Predicate`. That error, in [its entirety](#):

```
prog.cc:17:89: error: cannot call function 'Iterator find_if(Iterator, Iterator, F) [with Iterator = __gnu_cxx::__normal_iterator<std::string*>]
    find_if(vs.begin(), vs.end(), [](auto const& s) -> decltype(s == 2) { return s == 2; }); //2
                                                                    ^
prog.cc:11:10: note: constraints not satisfied
    Iterator find_if(Iterator b, Iterator e, F f);
    ^~~~~~
prog.cc:6:14: note: within 'template concept const bool Predicate [with F = main()::; T = {std::__iterator_traits<__gnu_cxx::__normal_iterator<std::string*>*>}>]
    concept bool Predicate = requires(F f, T... t) {
    ^~~~~~
prog.cc:6:14: note: with 'main():: f'
prog.cc:6:14: note: with 'std::__cxx11::basic_string& t#0'
prog.cc:6:14: note: the required expression 'f(t ...)' would be ill-formed
```

1.2.2. Concept overloading

The situation is no better when we try to overload with Concepts. Consider a seemingly innocuous example where we try to implement `on_each_month()`:

```
void on_each_month(Predicate<day_point> f);
void on_each_month(Predicate<std::string_view> f);
```

The intent of this function is to use the provided predicate to filter some set of dates. The first overload provides a `day_point` in the numeric format, the second is an optimized version that provides a serialized date. Let's try to use one:

```
on_each_month([](auto d) { return day_of_week(d) == friday; });
```

Even though we might perceive that one overload is viable, but the other not, this call will also fail to compile due to a hard error in attempting to instantiate the `string_view` overload (and again, with a very long list of diagnostics). This problem is the same fundamental problem demonstrated in the previous two examples. To fix this, the user either has to make the lambda non-generic, or constrain to only allow types that `day_of_week()` accepts, or copy the entire body into a trailing `decltype` expression. All of these solutions are repetitive and more involved than we thought we needed.

Neither the default `auto` return type, nor the expression-agnostic `decltype(auto)` as a trailing return type, are adequate solutions. To correctly solve the above issues, our initial wrapping of the overload set as:

```
[](auto&& arg) -> decltype(auto) { return foo(arg); }
```

needs to be extended out all the way to:

```
[](auto&& x) -> decltype(twice(std::forward<decltype(x)>(x))) noexcept(noexcept(twice(std::forward<decltype(x)>(x)))) { return twice(std::forward<decltype(x)>(x)); }
```

Needless to say, this is an unsatisfactory solution. Moreover, it may be insufficient.

1.3. `decltype(expr)` or `decltype((expr))`?

For the most part, there is no difference between `decltype(expr)` and `decltype((expr))`, they yield the same type for many expressions. The only cases where they yield *different* types are when `expr` is an *id-expression* or a class member access.

```
auto lam1 = [](auto&& x) -> decltype(x) { return x; };
auto lam2 = [](auto&& x) -> decltype((x)) { return x; };

int i;
lam1(i); // ok, returns int& because decltype(x) is int&
lam1(0); // error: can't bind lvalue to int&&
lam2(i); // ok, returns int& because x is an lvalue
lam2(0); // ok, returns int& because x is an lvalue

#define FWD(x) static_cast<decltype(x)&&>(x) // see p8644
struct S { int x; };
auto getx1 = [](auto&& s) -> decltype(FWD(s).x) { return FWD(s).x; };
auto getx2 = [](auto&& s) -> decltype((FWD(s).x)) { return FWD(s).x; };

S s(42);
getx1(s); // returns int
getx1(std::move(s)); // returns int
getx2(s); // returns int&
getx2(std::move(s)); // returns int&&
```

For those functions that simply return class member access, `decltype((expr))` is more likely to be the desired behavior. But this is a particularly subtle distinction.

1.4. Potential for improvement

In C++17 today, in order to lift an expression into a function, whether to be used as a callback or a predicate or a transformation, doing it properly is very difficult and very verbose. We need a trailing return type that wraps the whole expression, with an extra set of parentheses. We need a `noexcept` specification. This, in of itself, involves writing the function body in triplicate.

Furthermore, lambdas are unique in C++ in that they often are meaningless in a vacuum - they rely on the immediate context that comes from the expressions that contain them and code that precedes them. Since they often appear as sub-expressions, there is intrinsic value in making them shorter - if we can make them shorter by simply removing aspects of them that do not provide meaning to other readers of our code.

These issues arise in the context of function templates and member function templates as well. When the goal is to lift an expression into a function, the unnecessary boilerplate loses all expressiveness. This proposal seeks to eliminate this boilerplate in *simple* lambdas, function templates, and member function templates, without preventing users from writing the arbitrarily complex lambdas and functions that we can today.

2. Proposal

This paper proposes several different enhancements to the language as it relates to lambdas and functions. While we believe that all of these changes have benefits to beginners and experts alike, and we hope to see them all adopted, we understand that they are somewhat orthogonal of each other and in an effort to facilitate discussion and to provide EWG with options, we present them as as a primary proposal and several extensions to it.

2.1. Primary Proposal: `=> assignment-expression` for lambdas

This paper proposes the creation of a new function body introducer for lambdas, `=>`, which allows for a single *assignment-expression* as the body that will be its return statement. The expression will also be used to synthesize a SFINAE-, Concept-, and `noexcept`-friendly trailing return type. The *assignment-expression* will be used as body the body of the lambda and to determine a SFINAE- and `noexcept`-friendly return type.

That is, this lambda:

```
[&](auto&& a, auto&& b) => a.id() < b.id();
```

shall be treated as if it were written:

```
[&](auto&& a, auto&& b) -> decltype((a.id() < b.id())) noexcept(noexcept(a.id() < b.id())) { return a.id() < b.id(); }
```

Note that there is no `return` keyword used in the lambda body, it is unnecessary.

Some clarifying examples of the implications of the lambda body being an *assignment-expression*:

```
[&](auto&& x) => x[0](y) // lambda taking a single forwarding reference named x and returns the result of invoking x[0] with y
([&](auto&& x) => x[0])(y) // lambda taking a single forwarding reference named x and returning x[0], immediately invoked with y
// effectively, y[0]

f([&](auto&& x) => x, 4) // invoke f with two arguments: an identity lambda and 4
f([&](auto&& x) => (x, 4)) // invoke f with one argument: a lambda taking one argument and returning 4
```

While having the lambdas in this case have a return type of `auto` and no `noexcept`-specification is more consistent with the language as it stands, having simple lambdas and simple function templates have a return type of `decltype((expression))` and `noexcept`-specification of `noexcept(noexcept(expression))` saves the user from typing a bunch of repetitive boilerplate, and gets us to the point where we can just write *simple* code that just does the Right Thing™.

2.2. Extension: `=> assignment-expression` for functions

All the same motivation that applies to lambdas applies equally to functions. Hence, we propose that `=> assignment-expression` be allowed for all functions as well. This would allow, for instance, a simplification of `std::begin`:

```
template <class C> auto begin(C& c) -> decltype(c.begin()) { return c.begin(); } // C++14 today
template <class C> auto begin(C& c) => c.begin(); // this proposal
```

Except that now `begin` would additionally be conditionally `noexcept`.

2.3. Extension: Omitting types in lambda parameter declaration for abbreviated lambdas

With C++14, `auto&&` has become the safe default usage for lambdas. It's almost never wrong. Even if the lambda isn't intended to be used polymorphically, it's preferable to use `auto&&` rather than writing out the full name of the type. But once we're always writing `auto&&`, it itself doesn't actually have any meaning (if it ever did). Consider a predicate for sorting `lib::Widget`s by id:

```
// C++11
[](lib::Widget const& a, lib::Widget const& b) { return a.id() < b.id(); }

// C++14 - Widget is simply inferred
[](auto&& a, auto&& b) { return a.id() < b.id(); }
```

It's a full third shorter to just use `auto&&`, but what does `auto&&` gives us at that point? Moreover, in those cases where `auto&&` is wrong, it's not easy to distinguish. For instance, it may be important to only accept lvalues. But there is very little difference between a lambda taking its argument by `auto&` vs `auto&&`. Indeed, it looks more like a typo than a deliberate choice. It may also be important to accept arguments by reference to `const` (particularly where threading or COW is concerned). These situations are less common, but it's important to be able to visually distinguish between them in code - and the code we have today is often insufficiently different.

Hence, we propose that use of `=>` in lambdas, specifically, will allow for omitting types in the parameter list, defaulting to `auto&&`. This flips the treatment of a single identifier from identifying an unnamed argument of that type to identifying a forwarding reference with that name:

```
[(x, y) { ... } // lambda taking two unnamed parameters of types x and y
[(x, y) => (...)] // lambda taking two forwarding references named x and y
[(x&) => ...] // ill-formed

[(auto&&... xs) => f(xs...);] // primary proposal: lambda taking pack of forwarding references
[(xs...) => f(xs...);] // with this extension
```

Which allows the first example of this section to be rewritten as:

```
[(auto&& a, auto&& b) { return a.id() < b.id(); } // C++14
[(a, b) => a.id() < b.id()] // this proposal
```

While types *may* be omitted, it is not mandatory.

2.4. Implementation Experience

An implementation of this proposal for gcc has been graciously provided by Bastien Penavayre on [github](#), along with a version of the [compiler](#)

[explorer](#).

3. Examples

Transparently binding an overloaded member functions `func` to an instance `obj`:

```
// C++14
[&](auto&&... args) noexcept(noexcept(obj.func(std::forward<decltype(args)>(args)...)))
-> decltype((obj.func(std::forward<decltype(args)>(args)...))) {
    return obj.func(std::forward<decltype(args)>(args)...);
}

// this proposal
[&](args...) => obj.func(std::forward<decltype(args)>(args)...))

// with p0644
[&](args...) => obj.func(>>args...)
```

Sorting in decreasing order. Currently, we would rely on one of the many *named* function objects in the standard library to do this for us. While we recommend programmers use the standard algorithms, the standard function objects aren't quite on that level. This proposal allows us to just use `>`. It's longer, but arguably clearer:

```
std::sort(v.begin(), v.end(), std::greater{}); // C++17
std::sort(v.begin(), v.end(), [](x,y) => x > y); // this proposal
```

Once we move from directly working on the elements to working on other functions of the elements, the gain becomes much bigger. Sorting in decreasing order by ID is both shorter and much more readable:

```
std::sort(v.begin(), v.end(), [](auto&& x, auto&& y) { return x.id() > y.id(); }); // C++14
std::sort(v.begin(), v.end(), std::greater(), &lib::Widget::id); // Ranges TS, with projection*
// ... assuming id isn't overloaded
std::sort(v.begin(), v.end(), std::greater(), [](auto&& w) -> decltype(auto) { return w.id(); }); // Ranges TS, if id() is overloaded and returns
// an expensive-to-copy type. Don't forget decltype(auto)!
std::sort(v.begin(), v.end(), [](x,y) => x.id() > y.id()); // this proposal
```

Transforming a vector into another vector using a map as the function. Here, it's important to avoid the extra copy, and this proposal makes it easy to do so:

```
std::transform(v.begin(), v.end(), std::back_inserter(v2),
[&](auto&& key) { return map.at(key); }); // C++17: woops, extra copies!
std::transform(v.begin(), v.end(), std::back_inserter(v2),
[&](auto&& key) -> decltype(auto) { return map.at(key); }); // C++17
std::transform(v.begin(), v.end(), std::back_inserter(v2),
[&](key) => map.at(key)); // this proposal
```

Check if all of the elements have some predicate satisfied - by element directly:

```
std::all_of(v.begin(), v.end(), [](auto&& elem) { return elem.done(); }); // C++14, direct
std::all_of(v.begin(), v.end(), std::mem_fn(&lib::Element::done)); // C++14, with pointers to member functions*
std::all_of(v.begin(), v.end(), [](e) => e.done()); // this proposal
```

or on an external object:

```
std::all_of(v.begin(), v.end(), [&](auto&& elem) { return obj.satisfied(elem); }); // C++14, directly
std::all_of(v.begin(), v.end(), std::bind(&lib::Element::satisfied, std::ref(obj), _1)); // C++14, with std::bind*
std::all_of(v.begin(), v.end(), [&](elem) => obj.satisfied(elem)); // this proposal
```

Looking for an element by id:

```
auto it = std::find_if(v.begin(), v.end(), [&](auto&& elem) { return elem.id() == id; }); // C++14
auto it = std::find(v.begin(), v.end(), id, &lib::Widget::id); // Ranges TS, with projection*
auto it = std::find_if(v.begin(), v.end(), [&](elem) => elem.id() == id); // this proposal
```

Number of pairwise matches between two containers, using `std::inner_product` with two callables. With this proposal, it's actually a little longer, but importantly you don't have to know the names of things - you just write the expressions you want to write:

```
int matches = std::inner_product(as.begin(), as.end(), bs.begin(), 0, // C++14, example from cppreference.com
std::plus<>(), std::equal_to<>());
int matches = std::inner_product(as.begin(), as.end(), bs.begin(), 0, // this proposal
[](a,b) => a + b, [](a,b) => a == b);
```

Writing some overloads of getters:

C++17	This proposal
<pre>T& get() & { return value; } T const& get() const& { return value; } T&& get() && { return std::move(value); } T const&& get() const&& { return std::move(value); }</pre>	<pre>auto get() & => value; auto get() const& => value; auto get() && => std::move(value); auto get() const&& => std::move(value);</pre>

Writing a negating function object:

C++17	This proposal
<pre>template<typename F> class variadic_negator_functor { F f_; public: explicit variadic_negator_functor(F a_f) : f_(a_f) {} template<typename... Args> auto operator()(Args&&... args) -> decltype(!this->f_(std::forward<Args>(args)...)) noexcept(noexcept(!this->f_(std::forward<Args>(args)...))) { return !this->f_(std::forward<Args>(args)...); } template<typename... Args> auto operator()(Args&&... args) const -> decltype(!this->f_(std::forward<Args>(args)...)) noexcept(noexcept(!this->f_(std::forward<Args>(args)...))) { return !this->f_(std::forward<Args>(args)...); } // ... };</pre>	<pre>template<typename F> class variadic_negator_functor { F f_; public: explicit variadic_negator_functor(F a_f) : f_(a_f) {} template<typename... Args> auto operator()(Args&&... args) => !this->f_(std::forward<Args>(args)...); template<typename... Args> auto operator()(Args&&... args) const => !this->f_(std::forward<Args>(args)...); // ... };</pre>

Having to use nested lambdas. Dropping one `return` might not seem like much, until you need to do it multiple times, which comes up quite

often in functional programming. Consider this selection of examples from [Boost.Hana](#), courtesy of Louis Dionne (with some minor tweaks):

C++17	This proposal	This proposal + p0644
<pre>constexpr auto always = [] (auto x) { return [x(std::move(x))](auto const& ...y) -> decltype((x)) { return x; }; };</pre>	<pre>constexpr auto always = [] (auto x) => [x=std::move(x)](y...) => x;</pre>	<pre>constexpr auto always = [] (x) => [x==>x](y...) => x;</pre>
<pre>constexpr auto apply = [] (auto&& f, auto&& ...x) -> decltype(auto) { return std::forward<decltype(f)>(f)(std::forward<decltype(x)>(x)...); };</pre>	<pre>constexpr auto apply = [] (f, xs...) => std::forward<decltype(f)>(f)(std::forward<decltype(xs)>(xs)...);</pre>	<pre>constexpr auto apply = [] (f, xs...) => (>>f)(>>xs...);</pre>
<pre>constexpr auto compose2 = [] (auto f, auto g) -> decltype(auto) { return [f(std::move(f)), g(std::move(g))] (auto&& x, auto&& ...xs) -> decltype(auto) { return f(g(std::forward<decltype(x)>(x)), std::forward<decltype(xs)>(xs)...); }; };</pre>	<pre>constexpr auto compose2 = [] (auto f, auto g) => [f=std::move(f), g=std::move(g)](x, xs...) => f(g(std::forward<decltype(x)>(x)), std::forward<decltype(xs)>(xs)...);</pre>	<pre>constexpr auto compose2 = [] (f, g) => [f==>f, g==>g](x, xs...) => f(g(>>x), >>xs...);</pre>
<pre>constexpr auto partial = [] (auto f, auto ...x) { return [f(std::move(f)), x...](auto&& ...y) -> decltype(auto) { return f(x..., std::forward<decltype(y)>(y)...); }; };</pre>	<pre>constexpr auto partial = [] (auto f, auto... x) => [f=std::move(f), x...](y...) => f(x..., std::forward<decltype(y)>(y)...);</pre>	<pre>constexpr auto partial = [] (f, x...) => [f==>f, x==>x...](y...) => f(x..., >>y...); //with the addition of p0780, can forward the whole pack //instead of having to copy</pre>

In all of these cases, the function body - the expression we're trying to use - is really simple. Which is the point. Let's make it simpler to write simpler things.

4. Hyper-abbreviated lambdas

This proposal is about as abbreviated as you can get, without loss of clarity or functionality. But we can always go deeper. Any proposal on abbreviating lambdas would be incomplete without mentioning that numerous languages (including, but not limited to Swift, Elixir, and Scala), as well as the Boost.Lambda library, allow for writing expressions that themselves are callable. These refer to arguments by number and then synthesize a new closure. An example using Swift's syntax with the sorting by id predicate that we have been using throughout:

```
[](auto&& a, auto&& b) { return a.id() < b.id(); } // C++14 (51 characters)
[](a, b) => a.id() < b.id() // this proposal (28)
$1.id() < $2.id() // even more abbreviation (18)
```

or checking if an element satsifes an object's test:

```
[&](auto&& elem) { return obj.satisfies(elem); }; // C++14 (50 characters)
[&](elem) => obj.satisfies(elem); // this proposal (34)
obj.satisfies($1); // even more abbreviation (19)
```

While `$` directly is probably not the best choice in C++, there are other characters currently unusable in this context whose meaning we could overload here (e.g. `&`). In our opinion, this next step in abbreviation is unnecessary as this proposal gets you most of the way there and now we start moving towards a brevity that sacrifices some clarity.

Additionally, an earlier draft of this paper had proposed omitting the lambda capture entirely - with missing capture defaulting to `[&]`. This would allow the shortening the lambda `[&](e) => obj.satisfies(e)` to `(e) => obj.satisfies(e)` and possibly even to `e => obj.satisfies(e)`. It has been pointed out that this would be quite difficult to parse for compilers, and it has relatively marginal benefit compared to the other proposed changes, so it has been removed.

5. Prior Work and Effects on Existing Code

The original paper introducing what are now generic lambdas [2] also proposed extensions for omitting the *type-specifier* and dropping the body of a lambda if it's a single expression. This paper provides a different path towards those that same goal. Another paper proposing the same abbreviating syntax [4] has been merged into this one.

The usage of `=>` (or the similar `->`) in the context of lambdas appears in many, many programming languages of all varieties. A non-exhaustive sampling: C#, D, Erlang, F#, Haskell, Idris, Java, JavaScript, ML, OCaml, Swift. The widespread use is strongly suggestive that the syntax is easy to read and quite useful.

The sequence of characters `=>` can appear in code in rare scenarios, such as passing the address of the assignment operator as a template non-type argument: `x<&Y::operator=>`. However, such usage is incredibly rare, so this proposal would have very limited effect on existing code. Thanks to Richard Smith for doing a search.

6. Revision History

Since r0, this paper focused on a single syntax for abbreviating lambdas, and includes a stronger motivation for the need for SFINAE and `noexcept`. The section on abbreviated forwarding has been pulled out into its own paper [3], and a new section has been introduced discussing expression-based lambdas.

Since r1, this paper has combined with P0238 [4], changing the proposed SFINAE-friendly syntax to deducing `decltype(expr)` instead of `decltype(expr)` and also applying to functions. This proposal also drops omitting the capture syntax, and splits out each of the desired outcomes as extensions from the core proposal of `=> expr` for lambdas.

7. Acknowledgements and References

Thanks to Andrew Sutton for considering and rejecting several bad iterations of this proposal. Thanks to Richard Smith and Daveed Vandevoorde for looking into the practicality of this design along with parsing issues. Thanks to Nicol Bolas for refocusing the paper. Thanks to John Shaw for putting up with many crazy ideas.

Thanks especially to Adam Martin for presenting this proposal at Kona, and Nathan Myers for valuable feedback. Thanks to Casey Carter for presenting p0238 in Toronto and the valuable feedback from there.

Thanks to Bastien Penavayre for providing an implementation for gcc.

[1] [Overload sets as function arguments](#)

[2] [Proposal for Generic \(Polymorphic\) Lambda Expressions](#)

[3] [Forward without forward](#)

[4] [Return type deduction and SFINAE](#)